LIBYAN INTERNATIONAL MEDICAL UNIVERSITY (LIMU)

SOFTWARE ENGINEERING

CAPSTONE I

Milestone II (SRS):

Automated Financial Transaction Reconciliation System (AFTRS)

Amin Osman Student
ID no.: 4109
Fall Semester 2025/2026
Date of Submission: 25-01-2026

Contents

1. Overview

The Automated Financial Transaction Reconciliation System (AFTRS) is a web-based application designed to modernize the financial operations of different enterprise environments. The system replaces error-prone, manual spreadsheet comparisons with a deterministic C# reconciliation engine. It ensures data integrity through immutable audit logs, enhances efficiency via batch processing, and provides strategic foresight through budgeting and cash flow projection modules. This document outlines the requirements derived from the initial Milestone I analysis, adhering strictly to the Waterfall Model methodology and **IEEE Std 830-1998 [1]**..

2. Feasibility Study

2.1 Operational Feasibility Study

Unlike consumer applications that rely on user popularity, the operational feasibility of the AFTRS is defined by its ability to solve the critical "Bottleneck of Reconciliation" in corporate finance.

The feasibility analysis relied on **Domain Research** and **Process Efficiency Modeling**:

- **Current State Analysis:** Manual reconciliation of 1,000 transactions requires approximately 8–12 man-hours of skilled labor, costing the organization significant FTE (Full-Time Equivalent) resources.
- **Proposed State Analysis:** "Research into Robotic Process Automation (RPA) in banking **[4]** indicates that automated matching can reduce this processing time to under 2 hours."
- **Conclusion:** The system is operationally feasible because it drastically reduces operational costs and eliminates human error, providing an immediate Return on Investment (ROI) for the finance department.

2.2 Technical Feasibility Study

The proposed technology stack is mature, stable, and well-suited for enterprise applications:

- **Back-End:** ASP.NET Core MVC (C#) provides the robust, strictly typed logic required for financial calculations and the "Fuzzy Logic" matching algorithms.
- **Database:** SQL Server with Entity Framework Core allows for relational data integrity, complex queries for reporting, and secure storage of Audit and Security logs.
- **Front-End:** HTML, Tailwind CSS, and Chart.js ensure a responsive user interface capable of visualizing complex financial data without requiring high-performance client hardware.
- **Environment:** The system operates in a local-currency environment without the need for complex external API integrations, reducing technical risk.

2.3 Legal Feasibility Study

To ensure the proposed system adheres to financial regulations and data protection standards, **a consultation was conducted with a senior financial advisor and legal counsel.** The objective was to verify that automating the reconciliation process and storing financial data digitally does not violate any regulatory statutes.

The consultation confirmed that **there are no legal impediments** to the project, provided that specific compliance measures are implemented. Consequently, the AFTRS is designed with the following mandatory features to satisfy legal and contractual obligations:

- **Immutable Audit Trail:** The system's **Financial Audit Log** serves as the digital equivalent of a legal paper trail. It permanently records every manual resolution of a discrepancy, including the user's ID, timestamp, and a mandatory justification comment, ensuring no financial figure can be altered without a trace.
- **Data Privacy & Encryption:** In strict accordance with the system's **Terms and Conditions**, all sensitive user data (such as login credentials and personal identifiers) will be **encrypted** before storage. This ensures the system complies with data protection standards and safeguards user privacy against unauthorized access.
- **Accountability:** To comply with data access laws, the **Security Log** tracks every login and file upload attempt, ensuring that all actions can be attributed to a specific individual (Non-Repudiation).
- **Data Integrity:** The adoption of deterministic matching logic ensures that the system's output is consistent, reproducible, and verifiable, meeting the rigorous requirements of standard financial audits.

3. Source of requirements

Due to the specialized nature of financial accounting, the requirements were derived from **Domain-Specific Sources** rather than general user feedback:

1. **Legacy Data Artifacts(old physical files):** Analysis of actual "Internal Ledger" (CSV) and "Bank Statement" (Excel) files currently used by a company named Estishari. These documents dictated the input requirements and parsing logic.

   Why it's a source: The columns in those files (Date, Amount, Description) dictated exactly what columns your database needs. You didn't guess the columns; the files told you what they were.
2. **Regulatory Compliance Standards(rules of accounting and security):** The requirements for the "Financial Audit Log" and "Security Log" were derived from standard IT auditing principles (e.g., the principle of Non-Repudiation), ensuring the system meets legal financial scrutiny.

Why it's a source: This source is the reason we have the "Financial Audit Log" and "Security Log." we didn't build them for fun; we built them because the standards require them.

3. **Literature Review: Academic papers regarding 'Fuzzy Logic in Financial Matching' [3] provided the source**for the algorithm requirements in the Reconciliation Engine.

   Why it's a source: This justifies our "Fuzzy Logic" algorithm. We are proving that We read about other people solving this problem using algorithms, so we aren't just making up math.

4. Elicitation techniques

How did we extract the requirements from these sources?
To transform the high-level goal of automation into specific technical requirements, the following elicitation techniques were employed, focusing strictly on the internal resources and data available within the project scope:

4.1 Document Analysis (Internal Data Artifacts)

Instead of relying on external surveys, we analyzed the actual **legacy files** currently used by the financial department in Estishari company for manual reconciliation.

- **The Process:** We examined sample **Internal Ledger** (CSV) and **Bank Statement** (Excel) files we got from the accountant in the company.
- **The Findings:** This analysis revealed that the "Description" fields in these two files rarely match character-for-character (e.g., "مرتب للموظف كذا" vs. "إيداع مرتب لحساب كذا"). This discovery directly led to the requirement for **Fuzzy Logic matching** (FR-11) rather than simple exact matching. It also defined the specific column headers the system must recognize during parsing.

4.2 Interface Analysis (Current Workflow)

We analyzed the inputs and outputs of the current manual workflow to define the system boundaries.

- **Observation:** The current process involves manually exporting data from the accounting software and downloading a statement from the bank portal, then comparing them side-by-side in Excel.
- **Constraint Confirmation:** This confirmed that the AFTRS must operate as a **"Standalone File Processor"**. Since the system cannot connect to third-party bank APIs (out of scope), the interface must be designed to accept these specific file exports as manual uploads.

4.3 Expert Consultation (Brainstorming)

Sessions were conducted with the accountant of the Estishari company. These sessions focused on:

- **Compliance:** Determining exactly what data must be captured in the Audit Log to satisfy internal regulations.
- **Logic:** Defining the specific rules for when a transaction should be flagged as a "Discrepancy" versus a "Match." (eg. A cheque was given to an employee today and he deposited it tomorrow, in this situation the one day difference will considered as “match”, but if it was a direct transfer the one day difference will considered as “Discrepancy”)

5. System scenario

Scenario: The Month-End Reconciliation Process

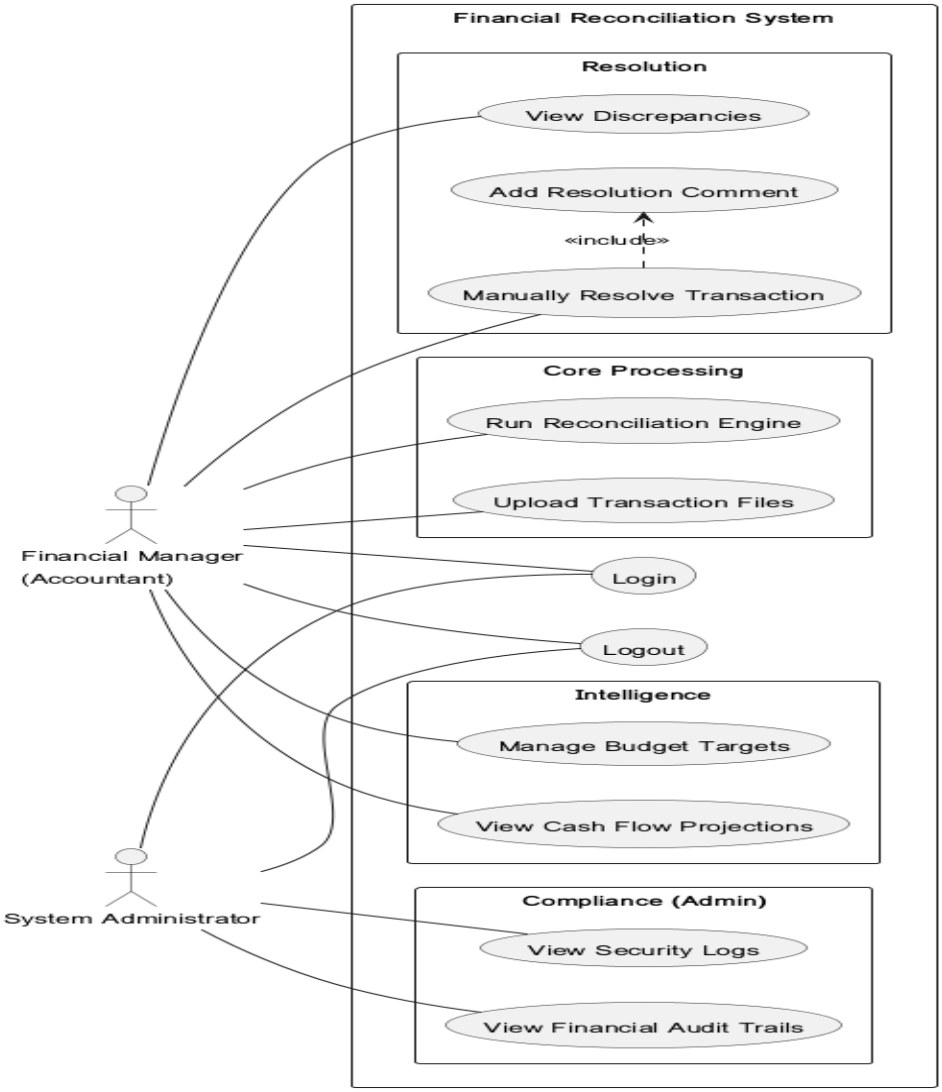
Mr. Amin, the Financial Manager, begins his end-of-month closing process. Traditionally, this required printing out bank statements and manually checking them against spreadsheet ledgers line-by-line. With the new system, Mr. Amin starts by securely logging into the AFTRS web application. Upon successful authentication, the system records his access in the Security Log and directs him to the main dashboard.

Instead of manual data entry, Amin navigates to the "Data Import" section. He uploads the company’s Internal Ledger (CSV) and the latest Bank Statement (Excel) provided by the bank. The system instantly parses these files, validating that the dates and amounts are in the correct format. With a single click, Amin triggers the **Reconciliation Engine**. The system automatically compares thousands of transactions using both exact matching for clear records and fuzzy logic to catch transactions with slightly different descriptions, completing a task that used to take hours in just a few minutes.

Once the automated processing is complete, the system presents a summary showing that 95% of transactions were matched automatically. However, five transactions are flagged as "Discrepancies." Amin enters the **Resolution Workflow** to investigate. He notices that one cheque has a typo in the reference number. He selects the corresponding items from the bank and ledger lists and manually links them. To comply with auditing regulations, the system prompts him to add a mandatory comment explaining the typo. Amin types "Reference number error by bank teller" and saves the match. This action is permanently recorded in the Financial Audit Log.

With the reconciliation complete and accurate, Amin moves to the **Strategic Planning** module. He views the "Cash Flow Projection" chart to analyze the company's spending trends for the upcoming quarter. Satisfied with the financial health of the organization, he exports the final "Reconciliation Status Report" as a PDF for the board meeting and securely logs out of the system.

6. Use case diagram



7. Use case tables

Use case-01: User Login

Use Case ID	UC-01
Title	User Login
Description	How the user accesses the system securely to perform their duties.
Actor	Financial Manager, System Administrator
Preconditions	• User has a valid account. • User is on the login page.
Success Postconditions	• User is authenticated. • Session is started and logged in Security Log. • User is redirected to the dashboard.
Main Success Scenario	1. User opens the application. 2. User enters Username and Password. 3. User clicks "Login." 4. System verifies credentials against the database. 5. System grants access.
Priority	High

Use case-02: User sign-up

Use Case ID	UC-02
Title	User Signup
Description	Creation of a new user account for a financial staff member.
Actor	Financial Manager
Preconditions	• User does not have an account.
Success Postconditions	• New account is created in the SQL database. • Default role is assigned.
Main Success Scenario	1. User clicks "Sign Up" on the landing page. 2. User enters Name, Email, and Password. 3. System validates password complexity. 4. System creates the record. 5. System redirects to Login.
Priority	Low

Use case-03: Upload internal ledger

Use Case ID	UC-03
Title	Upload Internal Ledger
Description	Importing the company's internal financial records.
Actor	Financial Manager
Preconditions	• User is logged in. • Valid CSV/Excel file is available.
Success Postconditions	• Ledger transactions are parsed and saved to the database.
Main Success Scenario	1. User navigates to "Import Data." 2. User selects "Internal Ledger" type. 3. User uploads the file. 4. System validates the file structure. 5. System saves the data.
Priority	High

Use case-04: Upload Bank Statement

Use Case ID	UC-04
Title	Upload Bank Statement
Description	Importing the external bank statement for comparison.
Actor	Financial Manager
Preconditions	• User is logged in. • Valid CSV/Excel file is available.
Success Postconditions	• Bank transactions are parsed and saved to the database.
Main Success Scenario	1. User navigates to "Import Data." 2. User selects "Bank Statement" type. 3. User uploads the file. 4. System validates the file structure. 5. System saves the data.
Priority	High

Use case-05: Run reconciliation engine

Use Case ID	UC-05
Title	Run Reconciliation Engine
Description	Triggering the algorithm to match internal and external transactions.
Actor	Financial Manager
Preconditions	• Both Ledger and Bank Statement files are uploaded.
Success Postconditions	• Transactions are marked as "Matched" or "Unmatched."
Main Success Scenario	1. User clicks "Reconcile Now." 2. System runs Exact Match logic (Amount, Date, ID). 3. System runs Fuzzy Logic (Description text). 4. System saves match status to database. 5. System displays summary.
Priority	High

Use case-06: View match result

Use Case ID	UC-06
Title	View Match Results
Description	Reviewing the list of matched transactions and pending discrepancies.
Actor	Financial Manager
Preconditions	• Reconciliation Engine has finished running.
Success Postconditions	• User views the status of all transactions.
Main Success Scenario	1. User navigates to "Results Dashboard." 2. System retrieves data from database. 3. System displays a grid of Matched and Unmatched items.
Priority	High

Use case-07: Manually resolve discrepancy

Use Case ID	UC-07
Title	Manually Resolve Discrepancy
Description	Manually forcing a match between two unmatched transactions.
Actor	Financial Manager
Preconditions	• Unmatched transactions exist in the system.
Success Postconditions	• Transactions are linked. • Action is logged in Audit Log.
Main Success Scenario	1. User selects an unmatched ledger item. 2. User selects the corresponding bank item. 3. User clicks "Manual Match." 4. System prompts for a comment (See UC-08). 5. System updates status to "Matched."
Priority	High

Use case-08: add audit comment

Use Case ID	UC-08
Title	Add Audit Comment
Description	Providing a mandatory justification for manual changes (Compliance).
Actor	Financial Manager
Preconditions	• User is in the process of Manual Resolution (UC-07).
Success Postconditions	• Comment is permanently saved in the Financial Audit Log.
Main Success Scenario	1. System displays comment modal/popup. 2. User types the reason (e.g., "Typo in ref number"). 3. User confirms. 4. System saves the text linked to the transaction.
Priority	High

Use case-09: Batch create template

Use Case ID	UC-09
Title	Batch Create Template
Description	Creating a rule to automatically handle recurring transactions (e.g., Monthly Rent).
Actor	Financial Manager
Preconditions	• User identifies a recurring transaction.
Success Postconditions	• A template is saved to automate future entries.
Main Success Scenario	1. User selects a transaction. 2. User clicks "Create Template." 3. User defines recurrence (e.g., "Monthly"). 4. System saves the template rule.
Priority	Medium

Use case-10: Trigger auto categorization

Use Case ID	UC-10
Title	Trigger Auto-Categorization
Description	Using the Heuristics Engine to assign categories (e.g., Utilities, Salary) to transactions.
Actor	Financial Manager
Preconditions	• Transactions are uploaded.
Success Postconditions	• Transactions have "Category" tags assigned.
Main Success Scenario	1. User clicks "Categorize Transactions." 2. System scans descriptions against keyword keywords. 3. System updates the "Category" field for each record. 4. System displays the categorized list.
Priority	Medium

Use case-11: Set budget target

Use Case ID	UC-11
Title	Set Budget Targets
Description	Defining the planned spending limits for specific categories.
Actor	Financial Manager
Preconditions	• Categories are defined.
Success Postconditions	• Budget limits are saved for the month.
Main Success Scenario	1. User navigates to "Budgeting." 2. User selects a category (e.g., "Marketing"). 3. User enters the budget amount (e.g., 5000 LYD). 4. User saves the target.
Priority	Medium

Use case-12: View budget report

Use Case ID	UC-12
Title	View Budget Report
Description	Comparing actual reconciled spending against the set budget.
Actor	Financial Manager
Preconditions	• Reconciliation is complete and Budgets are set.
Success Postconditions	• User views variance analysis (Actual vs. Budget).
Main Success Scenario	1. User navigates to "Budget Report." 2. System calculates totals per category. 3. System displays a bar chart of Spending vs. Limit.
Priority	Medium

Use case-13: View cashflow projection

Use Case ID	UC-13
Title	View Cash Flow Projections
Description	Visualizing future financial scenarios based on history.
Actor	Financial Manager
Preconditions	• Historical data exists.
Success Postconditions	• A projection graph is displayed.
Main Success Scenario	1. User navigates to "Cash Flow." 2. User selects projection period (e.g., 6 months). 3. System calculates trends. 4. System renders the line chart.
Priority	Medium

Use case-14: Export report

Use Case ID	UC-14
Title	Export Report
Description	Downloading financial data as PDF or Excel.
Actor	Financial Manager
Preconditions	• User is viewing a report (Reconciliation or Budget).
Success Postconditions	• File is downloaded to local device.
Main Success Scenario	1. User clicks "Export PDF." 2. System generates the document. 3. Browser initiates the download.
Priority	Medium

Use case-15: View security log

Use Case ID	UC-15
Title	View Security Log
Description	Reviewing login history and access attempts for security monitoring.
Actor	System Administrator
Preconditions	• User has Admin privileges.
Success Postconditions	• Security log table is displayed.
Main Success Scenario	1. Admin navigates to "Security Logs." 2. System retrieves login events (User, IP, Time). 3. System displays the list.
Priority	High

Use case-16: View audit trail

Use Case ID	UC-16
Title	View Audit Trail
Description	Reviewing the history of financial data modifications.
Actor	System Administrator
Preconditions	• User has Admin privileges.
Success Postconditions	• Audit log is displayed.
Main Success Scenario	1. Admin navigates to "Financial Audit Log." 2. System retrieves data changes (Who, What, When, Why). 3. System displays the immutable record.
Priority	High

1. Introduction:

This section provides an overview of the entire Software Requirements Specification (SRS) for the Automated Financial Transaction Reconciliation System (AFTRS).

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for the **Automated Financial Transaction Reconciliation System (AFTRS)**. This document states the whole aspects of constructing a web application for the purpose of simplifying the process of financial reconciliation of ledger account statement. It serves as the primary agreement and technical blueprint for the design and implementation phases of the Capstone I project, as well as it explains the web application user interfaces and their functions. Through This document a generic idea of this website is offered as an undergraduate project to implement in order to graduate and obtain a bachelor degree of software engineering LIMU university.

1.2 Scope

Product Name: Automated Financial Transaction Reconciliation System (AFTRS)

Product Description:
The AFTRS is a web-based financial management application designed to automate the comparison of internal company ledgers with external bank statements. By replacing manual spreadsheet processes with a deterministic C# matching engine, the system identifies discrepancies, enforces an auditable resolution workflow, and provides strategic financial insights.

In Scope (What the system will do):

- **Automated Reconciliation:** Parsing CSV/Excel files and matching transactions using exact and fuzzy logic.
- **Discrepancy Management:** A workflow for manually resolving unmatched transactions with mandatory audit comments.
- **Compliance:** comprehensive logging of all security events (login/access) and financial data modifications (Audit Log).
- **Strategic Planning:** Modules for Budgeting (Planned vs. Actual) and Cash Flow Projections (Visualizing future trends).

Out of Scope (What the system will NOT do):

- **Live API Integration:** The system will not connect directly to bank servers or third-party accounting software APIs; it relies strictly on file uploads.

- **Multi-Currency Support:** The system is limited to local currency operations and will not perform real-time exchange rate conversions.
- **ERP Functionality:** The system will not handle broader enterprise tasks such as payroll execution, inventory management, or invoicing.

Objectives:

- Reduce reconciliation time from days to under two hours per session.
- Ensure 100% auditability of financial adjustments.
- Transform financial data into visual business intelligence for strategic decision-making.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
AFTRS	Automated Financial Transaction Reconciliation System.
Reconciliation	The process of comparing two sets of records (internal ledger vs. bank statement) to ensure they are in agreement.
Fuzzy Logic	A computing approach based on "degrees of truth" rather than the usual "true or false" Boolean logic, used here for approximate string matching in transaction descriptions.
Ledger	The company's internal record of financial transactions.
RPA	Robotic Process Automation; technology that automates business processes.
FTE	Full-Time Equivalent; a unit that indicates the workload of an employed person.
EF Core	Entity Framework Core; the Object-Relational Mapper (ORM) used to interact with the SQL Server database.
MVC	Model-View-Controller; the architectural pattern used for the ASP.NET Core application.
CSV	Comma-Separated Values; a file format used for data import.

1.4 References

1. **IEEE Std 830-1998**: IEEE Recommended Practice for Software Requirements Specifications.
2. **Project Charter**: "Milestone I (Introduction): Automated Financial Transaction Reconciliation System (AFTRS)," Amin Osman, LIMU, Fall 2025/2026.
3. **Research**: *Machine Learning in Payment Reconciliation*, Journal of AI Research [JAIR].
4. **Research**: Azemi, E., et al. *Automating Financial Reconciliation: Leveraging RPA for Efficiency*, CEUR Workshop Proceedings.

1.5 Overview

The remainder of this document is organized as follows:

- **Section 2 (Overall Description)** describes the general factors that affect the product and its requirements, including the user interface, hardware constraints, and user characteristics.
- **Section 3 (Specific Requirements)** provides the detailed functional and non-functional requirements, including the specific inputs, outputs, and behavioral logic of the matching engine and reporting modules.

2. Overall Description

This section describes the general factors that affect the product and its requirements, including the user interface, hardware constraints, and user characteristics.

2.1 Product Perspective

The AFTRS is a standalone, web-based software solution designed to run on a local server environment within the university's financial department. It operates independently of external banking APIs, ensuring high security and data isolation.

2.1.1 System Interfaces

The system interacts with the following internal and external components:

- **Web Server (IIS):** The application is hosted on an Internet Information Services (IIS) server, serving the front-end to client browsers via HTTP/HTTPS.
- **Database Server:** The system interfaces with **Microsoft SQL Server** to store user credentials, audit logs, and transaction records. The application layer communicates with the database using **Entity Framework Core (EF Core)**.
- **File System:** The system interfaces with the local file storage to temporarily process uploaded CSV and Excel files before parsing them into the database.

2.1.2 User Interfaces

The user interface is designed as a responsive Web Application accessible via standard web browsers (Google Chrome, Microsoft Edge).

- **Dashboard Style:** The UI utilizes a modern "Dashboard" layout (using Tailwind CSS) with a sidebar navigation menu and a main content area.
- **Data Grids:** Financial data is presented in tabular grids with sortable and filterable columns.
- **Visualizations:** The Strategic Planning module utilizes **Chart.js** to render interactive line and bar charts for cash flow analysis.

2.1.3 Hardware Interfaces

- **Server Side:** The system requires a Windows Server environment capable of running .NET Core and SQL Server.
- **Client Side:** The system is hardware-agnostic regarding the client; it runs on any standard desktop or laptop with a minimum of 4GB RAM and a modern web browser.

2.1.4 Software Interfaces

- **Operating System:** Windows 10/11 or Windows Server 2019+ (Host).
- **Database System:** Microsoft SQL Server 2019 or later.
- **Framework:** ASP.NET Core 9.0 (MVC Pattern).
- **File parsers:** Libraries (such as CsvHelper) to interpret user-uploaded financial documents.

2.1.5 Communications Interfaces

- **HTTP/HTTPS:** The system uses standard Hypertext Transfer Protocol for client-server communication.

2.1.6 Memory Constraints

- **File Upload Limits:** To prevent memory overflow on the local server, individual file uploads (Ledgers/Bank Statements) are restricted to **25MB** per file.
- **Batch Processing:** The Reconciliation Engine processes transactions in batches of 1,000 records to maintain stable RAM usage during heavy calculation periods.

2.1.7 Site Adaptation Requirements

The system is designed for **Local Currency (LYD)** operations. The configuration settings allow the administrator to define specific "Tolerance Thresholds" (e.g., ignoring differences under 0.05 LYD) to adapt to the specific accounting practices of the deployment site.

2.2 Product Functions

The major functions of the AFTRS are categorized into four logical modules:

1. **Security & Compliance:**
    - Secure User Authentication (Login/Logout).
    - **Financial Audit Log:** Immutable recording of all manual modifications to transaction data.
    - **Security Log:** Tracking of all user access events.
2. **Core Reconciliation:**
    - Importing internal ledgers and bank statements (CSV/Excel).
    - **Automated Matching Engine:** Comparing transactions using Exact Logic (ID/Date/Amount) and Fuzzy Logic (Description similarity).
3. **Discrepancy Resolution:**
    - Visual interface for identifying unmatched transactions.
    - Workflow for manually linking transactions with mandatory justification comments.
4. **Strategic Intelligence:**
    - **Budgeting:** Setting monthly caps and monitoring actual spending.
    - **Cash Flow Projection:** Visualizing historical trends to forecast future cash positions.

2.3 User Characteristics

The system is designed for two specific classes of users:

User Class	Description	Technical skill level
Financial Manager (Accountant)	The primary user responsible for uploading files, running reconciliation, and analyzing reports. They are domain experts in accounting but may have limited technical skills.	**Intermediate:** Comfortable with web browsing and Excel/CSV handling. No coding skill required.
System Administrator	The IT specialist responsible for managing user accounts, monitoring security logs, and ensuring server uptime.	**Expert:** Knowledgeable in SQL, Server Management, and User Roles.

2.4 Constraints

- **Process Model:** The project strictly follows the **Waterfall Model**. All requirements and architectural designs must be finalized and documented before the Implementation (Coding) phase begins.
- **Regulatory Compliance:** The system must strictly adhere to the "Audit Trail" requirement. No financial record can be deleted from the database; "deleted" items must be soft-deleted (marked as inactive) to preserve history.
- **Currency Limitation:** The system is constrained to **Single Currency** operations. It does not support multi-currency conversion or live exchange rates.
- **Deterministic Logic:** The reconciliation algorithms must be deterministic (repeatable and verifiable). "Black box" AI models that cannot explain *why* a match was made are not permitted.

2.5 Assumptions and Dependencies

- **Assumption:** It is assumed that the bank statements provided by external banks are available in digital CSV or Excel formats.
- **Assumption:** The internal accounting department uses a standardized chart of accounts.
- **Dependency:** The system depends on the availability of the **Microsoft .NET Core Runtime** on the hosting server.

2.6 Apportioning of Requirements

Due to the time constraints of the Capstone I & II timeline, the following features are strictly **out of scope** for this version and are deferred to future releases:

1. **Direct API Integration:** Connecting directly to bank servers via Open Banking APIs.
2. **OCR Scanning:** Scanning physical paper receipts to convert them to digital data.
3. **Mobile Application:** A native mobile app for iOS/Android (the current system is a Responsive Web App only).

3 Specific Requirements

This section contains the detailed functional and non-functional requirements for the Automated Financial Transaction Reconciliation System (AFTRS).

3.1 Software Product Features Requirements

3.1.1 Feature 1: Authentication and Access Control

Purpose: To ensure that only authorized financial personnel and administrators can access the system and that their actions are tracked.
Use Cases: UC-01, UC-02, UC-15.

Functional Requirements:

- **FR-01:** The system shall provide a login interface requiring a unique Username and Password.
- **FR-02:** The system shall hash user passwords using standard cryptographic algorithms before storing them in the SQL database.
- **FR-03:** The system shall lock a user account after 5 consecutive failed login attempts to prevent brute-force attacks.
- **FR-04:** The system shall maintain a **Security Log** that records the Timestamp, User ID, IP Address, and Outcome (Success/Failure) for every login attempt.
- **FR-05:** The system shall enforce an automatic session timeout after 30 minutes of inactivity.

3.1.2 Feature 2: Data Import and Parsing

Purpose: To allow users to ingest external financial data (Bank Statements) and internal records (Company Ledgers) into the system for processing.
Use Cases: UC-03, UC-04.

Functional Requirements:

- **FR-06:** The system shall accept file uploads in **.CSV** (Comma Separated Values) and **.XLSX** (Excel) formats.
- **FR-07:** The system shall validate that uploaded files contain the mandatory columns: Transaction Date, Description, Reference Number, and Amount.
- **FR-08:** The system shall reject files exceeding **25MB** in size and display an error message to the user.
- **FR-09:** Upon successful upload, the system shall parse the records into the Staging_Transactions database table and display a summary count of loaded records.
- **FR-09a (File Validation):** The system shall verify if a file has already been processed by comparing the file name and the cryptographic hash (MD5/SHA-256) of the uploaded document. If a duplicate is detected, the system shall alert the user and block the upload.
- **FR-09b (Transaction Validation):** During parsing, the system shall check individual transaction IDs against existing records in the database. If a transaction with the same **Reference Number, Date, and Amount** already exists, the system shall skip that specific row to prevent data redundancy.

3.1.3 Feature 3: The Reconciliation Engine

Purpose: To automate the comparison of transactions between the Internal Ledger and the Bank Statement.
Use Cases: UC-05, UC-06, UC-10.

Functional Requirements:

- **FR-10:** The system shall execute **Level 1 Matching (Exact Match)**: Transactions are automatically matched if Date, Amount, and Reference Number are identical in both files.
- **FR-11:** The system shall execute **Level 2 Matching (Fuzzy Logic)** for remaining unmatched items: The system shall match transactions if Amount is identical and the Description text has a string similarity score of >80% .
- **FR-12:** The system shall tag matched transactions with a status of Reconciled and unmatched transactions with a status of Discrepancy.
- **FR-13:** The system shall run the **Heuristics Engine** to assign a Category (e.g., "Utilities", "Payroll") to transactions based on keyword analysis of the Description field.

3.1.4 Feature 4: Discrepancy Resolution Workflow

Purpose: To provide a controlled, auditable process for fixing transactions that the engine could not match automatically.
Use Cases: UC-07, UC-08, UC-16.

Functional Requirements:

- **FR-14:** The system shall provide a "Discrepancy Dashboard" displaying side-by-side lists of unmatched Ledger and Bank items.
- **FR-15:** The system shall allow the user to manually select one Ledger item and one Bank item to force a match.
- **FR-16: Mandatory Audit:** The system shall require the user to enter a text justification (Comment) before saving a manual match.
- **FR-17:** The system shall record the Manual Match event in the **Financial Audit Log**, capturing: Old Status, New Status, User ID, Timestamp, and Justification Comment. This log cannot be deleted or modified by standard users.

3.1.5 Feature 5: Strategic Financial Planning

Purpose: To transform historical data into future insights.
Use Cases: UC-11, UC-12, UC-13.

Functional Requirements:

- **FR-18:** The system shall allow users to define monthly **Budget Limits** for specific categories (e.g., "Marketing Limit = 5000 LYD").
- **FR-19:** The system shall generate a **Budget vs. Actual Report** highlighting categories where spending exceeded the defined limit.
- **FR-20:** The system shall generate a **Cash Flow Projection Graph** (Line Chart) using Chart.js, projecting the next 3 months of balance trends based on the average spending of the previous 6 months.

3.2 External Interface Requirements

3.2.1 User Interfaces

- The system shall utilize a responsive Web Interface built with **HTML5** and **Tailwind CSS**.
- The "Reconciliation View" shall use color-coding: **Green** for Matched, **Red** for Unmatched, and **Yellow** for Manually Resolved items.

the following wireframes describes how the system is imagined :

A. Authentication

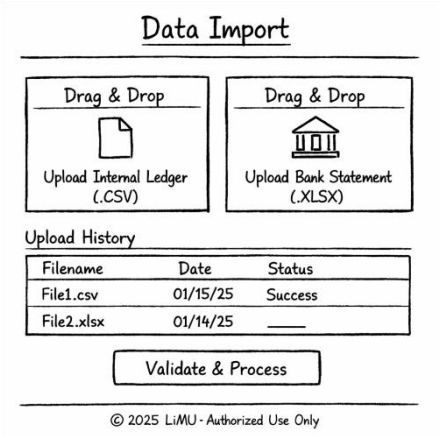
A.1 Secure login:

- **Primary Actor:** All Users (Financial Manager, System Administrator)
- **Description:** The centralized entry point for the application. It enforces Access Control (Feature 1) by requiring valid credentials. The footer "Authorized Use

Only" reinforces the security constraints, and the system performs password hashing upon submission.

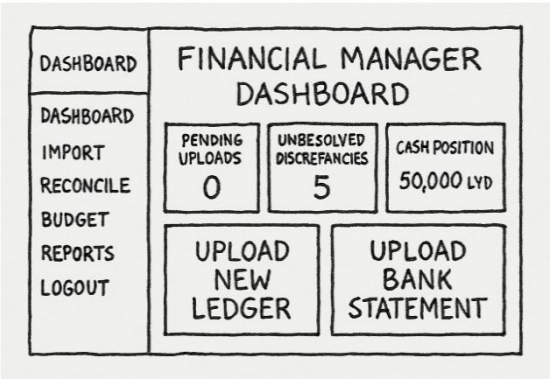
B. Core Processing (Financial Manager)

B.1 Data import:



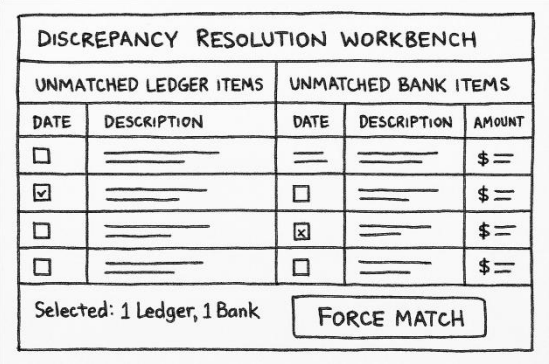
- **Primary Actor:** Financial Manager
- **Description:** This screen handles Feature 2: Data Import. It provides two distinct drag-and-drop zones to ensure the user separates the "Internal Ledger" (CSV) from the "Bank Statement" (Excel/CSV). The "Upload History" grid at the bottom provides immediate feedback on file validation status (FR-09).

B.2 Financial Manager Dashboard:



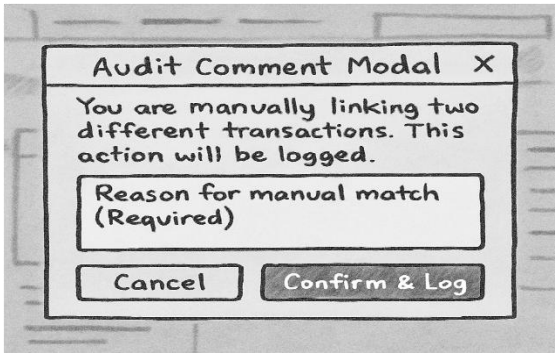
- **Primary Actor:** Financial Manager
- **Description:** The main operational hub. It displays real-time statistics regarding pending uploads and unresolved discrepancies, serving as the starting point for the Monthly Reconciliation Scenario. The sidebar provides navigation to all functional modules.

B.3 Discrepancy Resolution Workbench:



- **Primary Actor:** Financial Manager
- **Description:** The core interface for Feature 4: Discrepancy Resolution. It utilizes a "Split-Screen" layout (FR-14) to show unmatched Ledger items on the left and unmatched Bank items on the right. This allows the accountant to visually compare and select transactions for manual matching.

B.4 Audit Comment Modal:



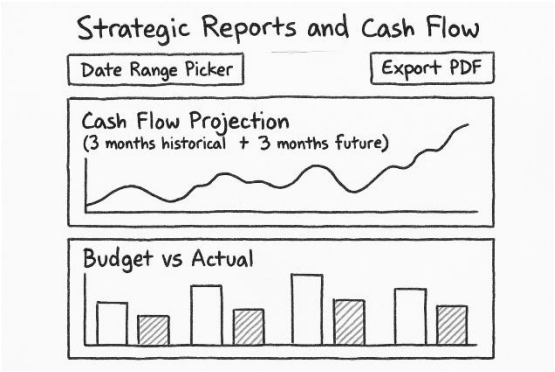
- **Primary Actor:** Financial Manager
- **Description:** A mandatory compliance popup that appears when a user attempts a "Force Match." It enforces FR-16 (Mandatory Audit) by requiring the user to type a justification (e.g., "Typo in Ref") before the system saves the change to the database.

C. Strategic Intelligence

C.1 Monthly Budget Configuration:

- **Primary Actor:** Financial Manager
- **Description:** The input interface for Feature 5: Strategic Planning. It allows the user to define specific monetary limits (LYD) for different expense categories (e.g., Marketing, Payroll), which drives the variance analysis reports.

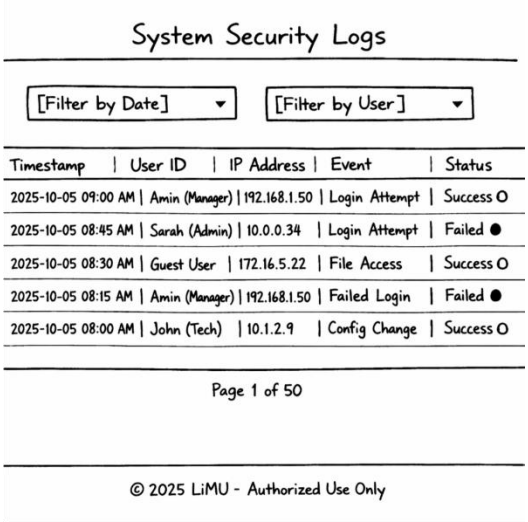
C.2 Strategic Reports & Cash Flow:



- **Primary Actor:** Financial Manager
- **Description:** The output interface for business intelligence. It visualizes data using Chart.js (FR-20). The top Line Chart displays historical and projected cash flow, while the bottom Bar Chart compares Actual Spending against the Budget Targets set in Figure 3.6.

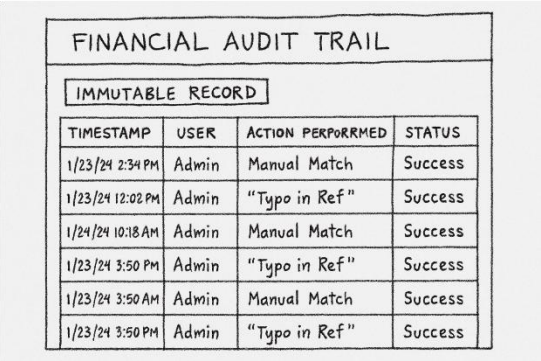
D. Admin & Compliance

D.1 System Security Logs:



- **Primary Actor:** System Administrator
- **Description:** A monitoring tool for Feature 1: Security. It displays a filterable list of all access attempts, recording the Timestamp, User ID, IP Address, and Status (Success/Failure) as required by FR-04 to ensure accountability.

D.2 Financial Audit Trail:



- **Primary Actor:** System Administrator
- **Description:** The immutable record of all data modifications (Feature 4). It provides a read-only view of manual actions taken by the Financial Manager, displaying the specific "Action Performed" and the corresponding "Justification," ensuring the system meets Legal Feasibility requirements.

3.2.2 Software Interfaces

- **Input Data Format:** The system interfaces with user-provided CSV files. The expected structure is:
    - Date (Format: DD/MM/YYYY)
    - Description (String, Max 200 chars)
    - Amount (Decimal, 2 precision)
    - Type (Credit/Debit)
- **Database Interface:** The system interfaces with **Microsoft SQL Server** using **Entity Framework Core** for all data storage and retrieval.

3.3 Performance Requirements

- **PR-01:** The Reconciliation Engine shall process a batch of 1,000 transactions in under **60 seconds**.
- **PR-02:** All web pages shall load in under **2 seconds** on a standard local network connection.
- **PR-03:** The total time to complete a full monthly reconciliation session (Upload + Match + Review) shall not exceed **2 hours** .

3.4 Design Constraints

- **DC-01:** The system must be developed using the **ASP.NET Core MVC** framework (C#).
- **DC-02:** The system must use **Deterministic Logic** for financial calculations; no non-verifiable AI models are permitted for the core math.
- **DC-03:** The system must operate strictly in **Local Currency (LYD)**; multi-currency conversion is restricted.

3.5 Software System Attributes

3.5.1 Reliability

- The system shall ensure data accuracy by using the Decimal data type in C# for all financial values to avoid floating-point rounding errors.

3.5.2 Availability

- The system is designed for business-hours availability (9:00 AM – 5:00 PM). It does not require 24/7 uptime redundancy for this phase.

3.5.3 Security

- **Data Integrity:** The Financial Audit Log must be **immutable** (Read-Only). No user, including the Administrator, shall have a UI option to delete audit records.
- **Access Control:** The system shall implement Role-Based Access Control (RBAC) distinguishing between Admin (System Config) and User (Financial Operations).

3.5.4 Maintainability

- The code shall follow the **MVC Pattern** (Model-View-Controller) to separate business logic from the user interface, ensuring that future interface changes do not break the calculation logic.

3.6 Other Requirements

- **Documentation:** The system shall include an embedded "Help" page explaining the correct format for CSV uploads.